

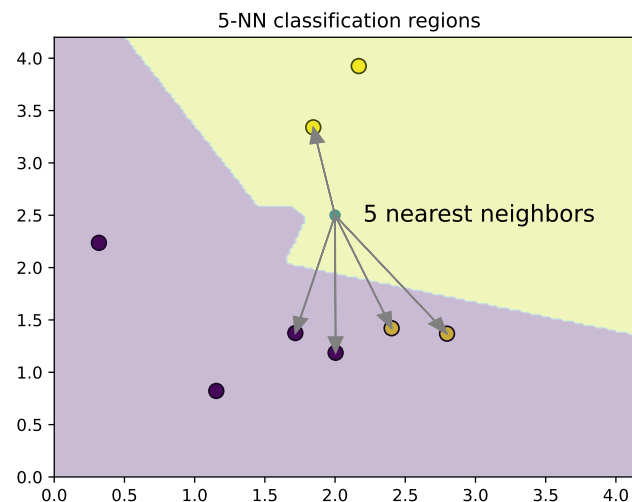
Machine Learning in Practice I

Lecture 7

Antal Jakovac, 2025

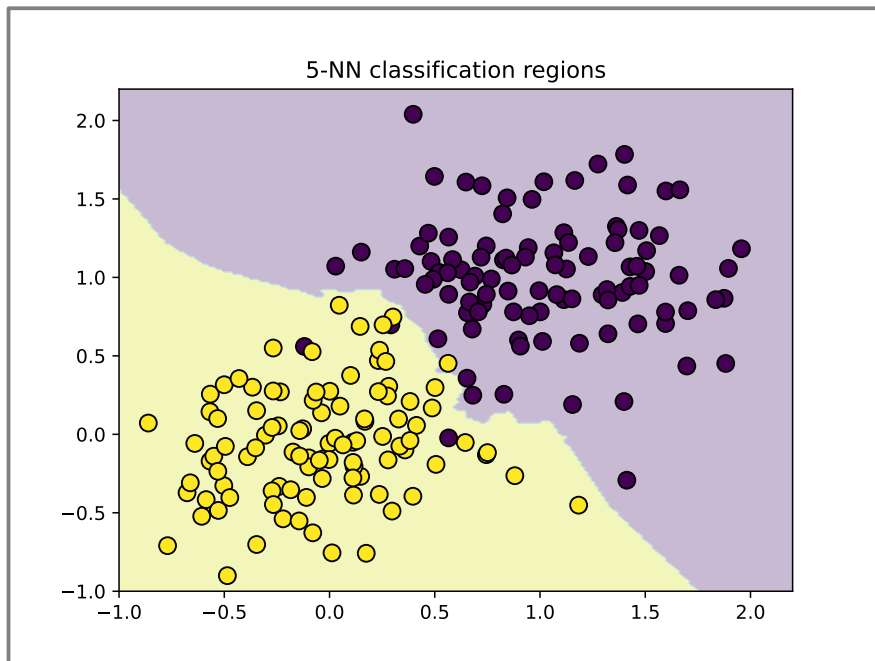
k-nearest neighbour (KNN) model

- a different idea: similar points are close in the feature space
- for a good guess on the class peek the neighbouring points
- algorithm:
 - keep all points in the training set
 - new point: compute all the distances
→ needs a distance measure
 - keep first k closest points
 - estimated class: majority vote



-  pros
 - ➔ simple, intuitive, no training phase
 - ➔ naturally handles multi-class problems
-  cons:
 - ➔ slow at prediction time (needs to compare with all samples)
 - ➔ sensitive to irrelevant features and scaling (c.f. distance measure)
 - ➔ struggles in high dimensions (“curse of dimensionality”)

- in code



```
# Random Forest example  
from sklearn import svm  
from sklearn.neighbors import KNeighborsClassifier
```

```
# Train a 3-NN classifier  
knn = KNeighborsClassifier(n_neighbors=5)  
knn.fit(X,y)
```

```
# Predict on the test set  
y_pred = knn.predict(X_test)
```

```
# Evaluate  
acc = accuracy_score(y_test, y_pred)
```

→ acc = 97.5%

Estimate relevance of features



- not all features contribute equally to the result – to reduce the number of features we shall assess their relevance
- idea: measure the change in the result when change the feature
- relevance measures (the most used ones)
 - ➔ feature importance (only for random forest)
 - ➔ permutation importance, feature masking
 - ➔ mutual information
 - ➔ in deep neural networks Gradient-weighted Class Activation Mapping (Grad-CAM)



- we will demonstrate the feature relevance measures in a 2-class 20 dimensional dataset with 2000 elements

```
# make multidimensional dataset  
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
  
ndim = 20  
Nsamples = 2000  
  
X, y = make_classification(n_samples=Nsamples, n_features=20,  
                           n_classes=2, random_state=42)  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
                                                    random_state=42)
```

- works only for random forest
- idea:
 - measure the impurity (e.g. Gini impurity) of all nodes before and after a decision
 - if we use a feature for a decision, measure the impurity decrease

$$\text{Importance}(f) = \sum_{\text{nodes using } f} \text{Impurity decrease at that node}$$

- average over the forest

$$\text{Importance}_{\text{forest}}(f) = \frac{1}{N_{\text{trees}}} \sum_{t=1}^{N_{\text{trees}}} \text{Importance}_t(f)$$

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier

```
# make mulitdimensional dataset  
from sklearn.ensemble import RandomForestClassifier  
  
# Creating a Random Forest classifier  
clf = RandomForestClassifier(n_estimators=100, random_state=42)  
  
# Training the classifier on the training data  
clf.fit(X_train, y_train)
```

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result

```
# evaluate result  
from sklearn.metrics import confusion_matrix  
  
# Making predictions on the testing data  
y_pred = clf.predict(X_test)  
  
# confusion matrix  
cm = confusion_matrix(y_test, y_pred)  
  
accuracy = np.trace(cm) / np.sum(cm)  
  
→ accuracy = 93.5%
```

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result
- get feature importance

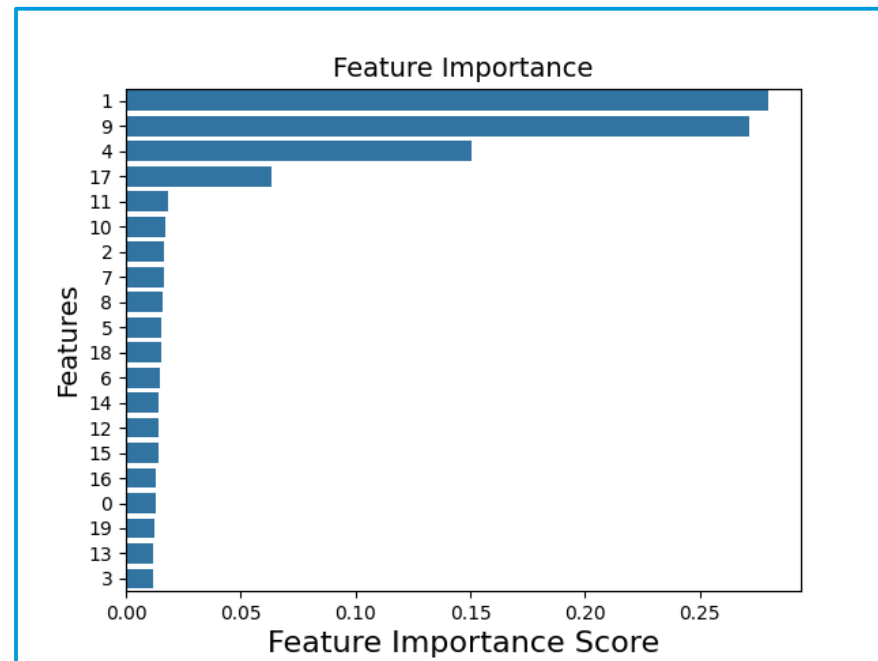
```
feature_importance = clf.feature_importances_  
  
→ np.array([0.01291599, 0.27999269, 0.01658218,  
0.01157727, 0.15054909, 0.01555601, 0.0150478 ,  
0.01637177, 0.01582075, 0.27170634, 0.01723006,  
0.01812296, 0.01401398, 0.01165928, 0.01404261,  
0.01398676, 0.01323008, 0.0636087 , 0.01551626,  
0.01246943])
```

Feature importance

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result
- get feature importance
- visualize it

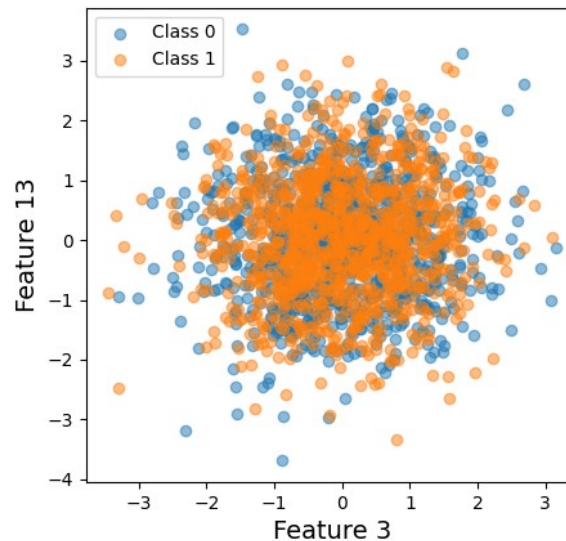
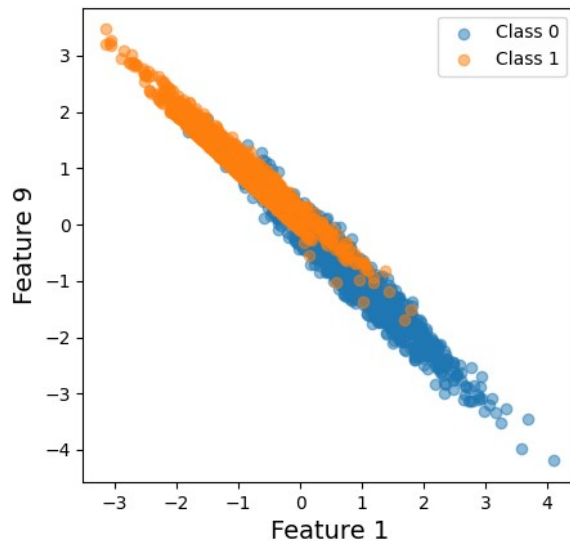
```
sorted_idx = np.argsort(feature_importance)[::-1]

plt.figure()
sns.barplot(
    x=feature_importance[sorted_idx],
    y=np.arange(len(sorted_idx)),
    orient='h')
plt.xticks(np.arange(len(sorted_idx)),sorted_idx)
```



Feature importance

- example: create a 20 dimensional dataset with 2000 elements
- most (1,9) and least (13,3) important features





- works for all methods
 - idea:
 - train the model, measure the performance – baseline
 - for each feature permute the values among samples
 - measure the performance again
 - relevance of the feature: drop in performance
- $$\text{Importance}(f) = \text{Score}_{\text{original}} - \text{Score}_{\text{permuted}}(f)$$
- instead permutation we can use a constant value (e.g. average), or a random value → mask that feature

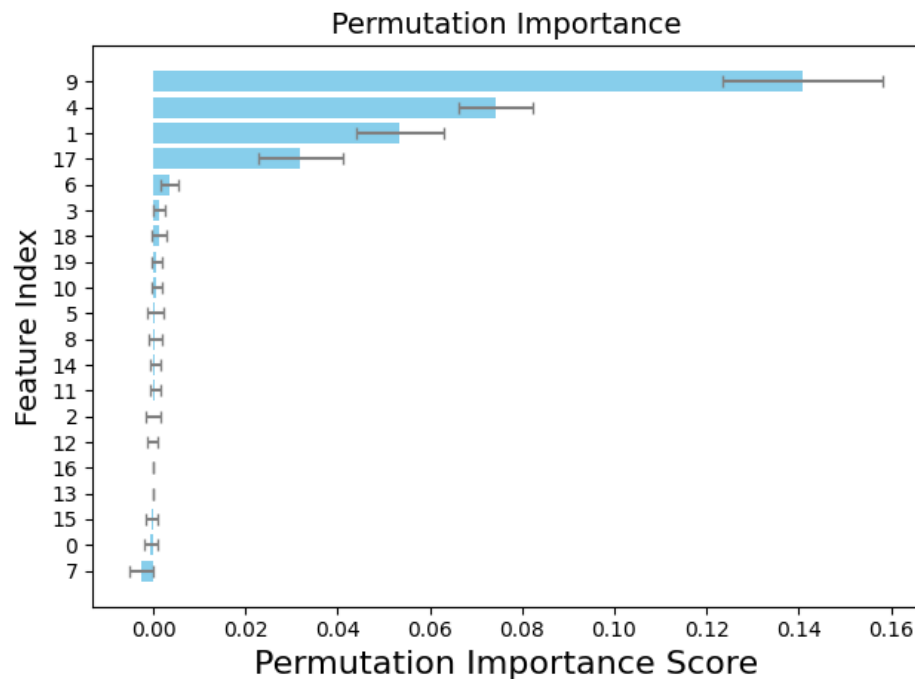


- get permutation importance in Python

```
# import package  
from sklearn.inspection import permutation_importance  
  
# use trained model clf to calculate importance  
result = permutation_importance(clf, X_test, y_test,  
                                n_repeats=10, random_state=42)  
  
# read mean and variance of the estimates  
importances = result.importances_mean  
std = result.importances_std
```

Permutation importance

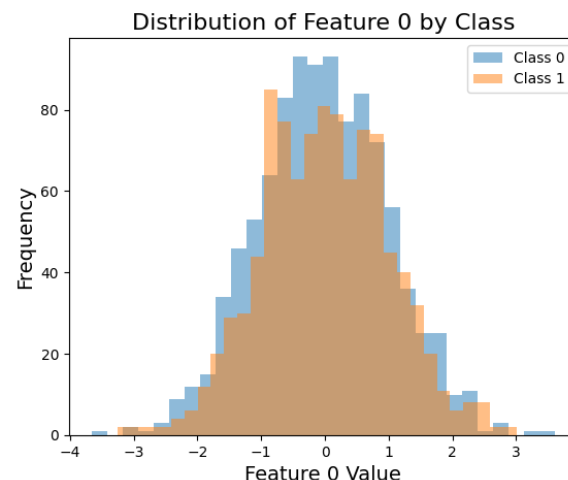
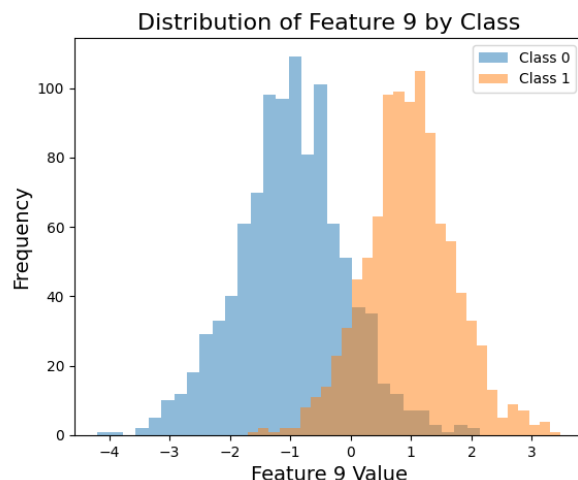
- get permutation importance in Python
- visualize the result
- same features are selected as before, but order is different



Permutation importance

- choose feature 9, and feature 0 to demonstrate the effect
- plot the distribution of the given feature, conditioned for the class (histogram)

$$P(\text{feature value} = f \mid \text{class})$$



- can be used for any models – for any two set, actually
- idea:
 - calculate the joint probability of two sets $p(x \in X, y \in Y)$
 - if x and y are independent, then the joint probability is the product

$$p(x, y) \xrightarrow{\text{independent}} p_X(x) p_Y(y)$$

- measure the dependence (mutual information) by

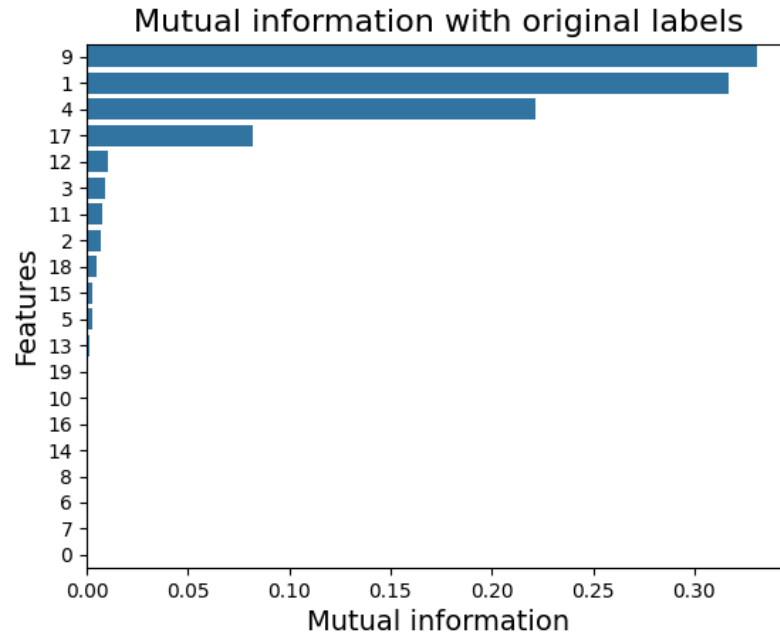
$$I(X, Y) = \sum_{x \in X, y \in Y} p(x, y) \ln \left(\frac{p(x, y)}{p_X(x) p_Y(y)} \right)$$

- can be used to measure mutual information between features and labels

- in Python there is a package to calculate mutual information
- the code

```
# import package  
from sklearn.feature_selection import mutual_info_classif, mutual_info_regression  
  
# we can use the original labels, or the model-predicted ones  
# characteristic for both the system and the trained model  
# y = clf.predict(X)  
  
mi_model = mutual_info_classif(X, y, random_state=42)  
mi_model /= np.sum(mi_model)
```

- in Python there is a package to calculate mutual information
- the code
- visualization
- similar result as before



END OF LECTURE 7